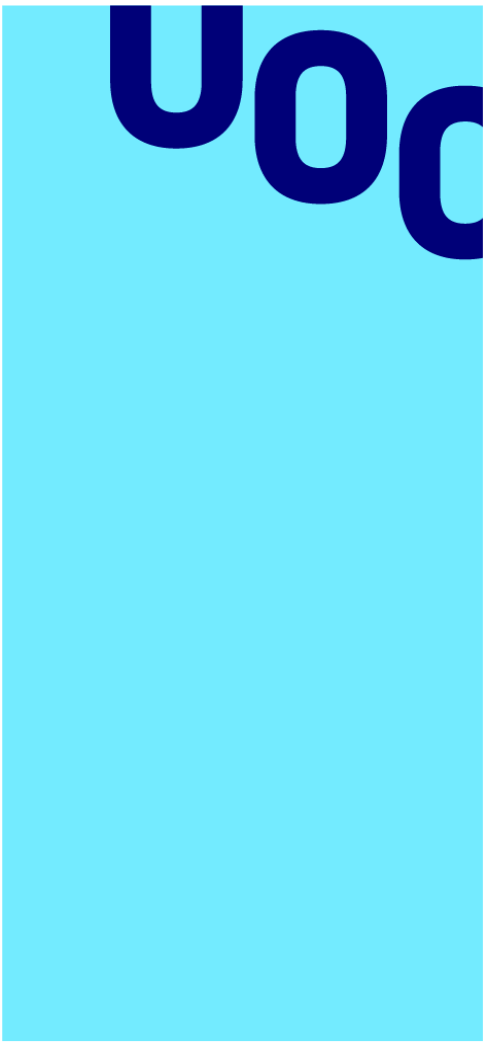
collat

PEC 1

Empezando...



Universitat Oberta de Catalunya

Información relevante:

- Fecha límite de entrega: 06/10/2026
- Peso en la nota de PECs: 10%.

Contenido

Información docente **2**
Prerrequisitos 3
Objetivos 3
Resultados de aprendizaje 3
Enunciado **4**
Tarea 1 - Configuración del entorno básico 4
Tarea 2 - Configurando IntelliJ 6
Ejercicio 1 - Debugging (2 puntos) 11
Tarea 3 - Testing con JUnit 13
Ejercicio 2 (3 puntos) 18
Ejercicio 3 (5 puntos) 20
Formato y fecha de entrega **24**
Anexo: Problemas frecuentes **25**
Uso de servicios de almacenamiento en la nube 25



En esta actividad no está permitido el uso de herramientas de inteligencia artificial.

En el plan docente y en la web sobre integridad académica y plagio de la UOC encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.

Información docente

Esta PEC está vinculada al módulo teórico "Introducción al paradigma de la programación orientada a objetos" de los apuntes de la asignatura. Léelo antes de empezar la PEC. También debes leer el capítulo 2 de la Guía de Java.

Prerrequisitos

Para hacer esta PEC necesitas saber conceptos básicos de algorítmica (aprendidos en la asignatura anterior).

- Conocer el paradigma de la programación estructurada.
- La sintaxis de un lenguaje imperativo de uso común, p.ej. C, Python, etc.
- Declaración y uso de variables de tipo primitivo (`int`, `float`, etc.).
- Bucles (`while`, `for` y `do-while`).
- Condicionales (`if-else` y `switch`).
- Uso de arrays.
- Creación y uso de funciones (en nuestro contexto llamadas “métodos”).

Objetivos

Con esta PEC el equipo docente de la asignatura busca que:

- Uses Java como un lenguaje de programación imperativo y así conozcas la sintaxis básica de este lenguaje.
- Te familiarices con la dinámica de la asignatura: metodología, tipología de enunciados, nomenclaturas, etc.

Resultados de aprendizaje

Con esta PEC debes demostrar que eres capaz de:

- Configurar un entorno de trabajo básico para programar con Java: JDK y Notepad++.
- Implementar programas sencillos en Java basados en el paradigma de la programación estructurada.
- Entender qué hace un programa o código dado por un tercero.
- Configurar y usar con cierta soltura un entorno de desarrollo integrado (IDE) como IntelliJ.
- Ser capaz de depurar (*debug*) en un IDE un programa con errores para corregirlo.
- Utilizar test unitarios para determinar que un programa es correcto.

Enunciado

Esta PEC contiene 3 tareas (actividades no evaluables) y 3 ejercicios (sí evaluables). Tienes que entregar tu solución de los 3 ejercicios evaluables (ver el último apartado).



Debido a que las actividades están encadenadas (i.e. para hacer una se debe haber comprendido la anterior), **es altamente recomendable hacer las tareas y ejercicios en el orden en que aparecen en este enunciado.**

Tarea 1 - Configuración del entorno básico

Antes de empezar, debes:

- Instalar el JDK en tu ordenador. Para ello, lee el apartado "1.1. Instalando JDK" de la Guía de Java. **Recomendamos instalar JDK 21**. Además, se recomienda leer el apartado 1.2 de la Guía de Java.
- Usar un editor de texto plano (p. ej., el que viene con tu sistema operativo): vim, notepad, TextEdit, etc. No obstante, para Windows recomendamos **Notepad++**.

Una vez realizados los pasos anteriores, puedes hacer la siguiente tarea para comprobar que el entorno básico para desarrollar programas en Java lo tienes bien instalado. Para ello, sigue estos pasos:

1. Abre el directorio `PAC1Task1` que te proporcionamos con el enunciado.
2. Allí encontrarás un fichero llamado `PAC1Task1.java`.
3. Abre una terminal o procesador de comandos (cmd).
4. En el terminal, ubícate en el directorio `PAC1Task1` y, una vez en él, ejecuta el comando:

   `javac PAC1Task1.java`
5. Si todo va bien, en el terminal no sucederá absolutamente nada. Esto quiere decir que el fichero `PAC1Task1.java` no ha producido ningún error. Sin embargo, si miras dentro del directorio `PAC1Task1`, debería haberte aparecido un nuevo fichero llamado `PAC1Task1.class`. Este nuevo fichero es el *bytecode* de `PAC1Task1.java`.
6. A continuación, vamos a ejecutar `PAC1Task1.class`. Para ello, ejecuta el siguiente comando en el terminal que tienes abierto (que está ubicado en el directorio `PAC1Task1`):

   `java PAC1Task1`

7. Verás que el programa escribe el siguiente mensaje: `No data`

8. Ahora ejecuta el programa `PAC1Task1` pasándole argumentos. Por ejemplo:

```
java PAC1Task1 5 -4 12 3 -20
```

9. La ejecución anterior debería producir la siguiente salida por pantalla:

```
-20   12    -0.8
```

10. Haz más pruebas pasándole diferentes argumentos. Después abre el fichero `PAC1Task1.java` y observa el código que contiene. Asimismo, prueba otras ejecuciones del programa. ¿Qué crees que hace este programa?

11. Respuesta:

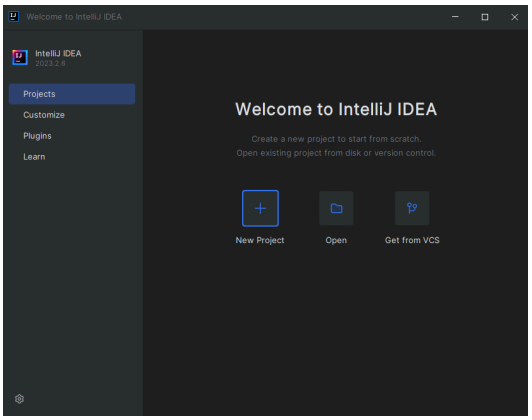
Este programa muestra por pantalla, en función del listado de números facilitados, (1) el valor mínimo del listado, (2) el valor máximo, y (3) el valor medio. Si no se facilita ningún número, se muestra el mensaje "No data".

Tarea 2 - Configurando IntelliJ

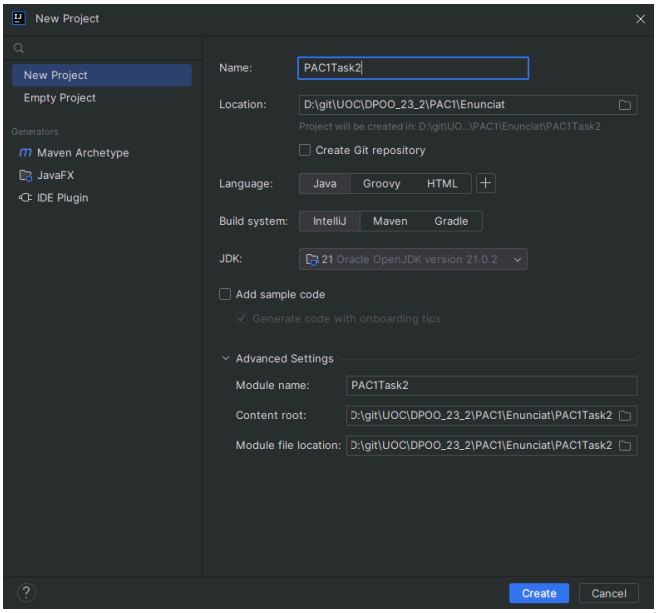
Antes de continuar, debes:

> Instalar y configurar el IDE IntelliJ en tu ordenador. Para ello, lee el documento **Guía_IntelliJ_IB.pdf** que encontrarás en la actividad del aula.

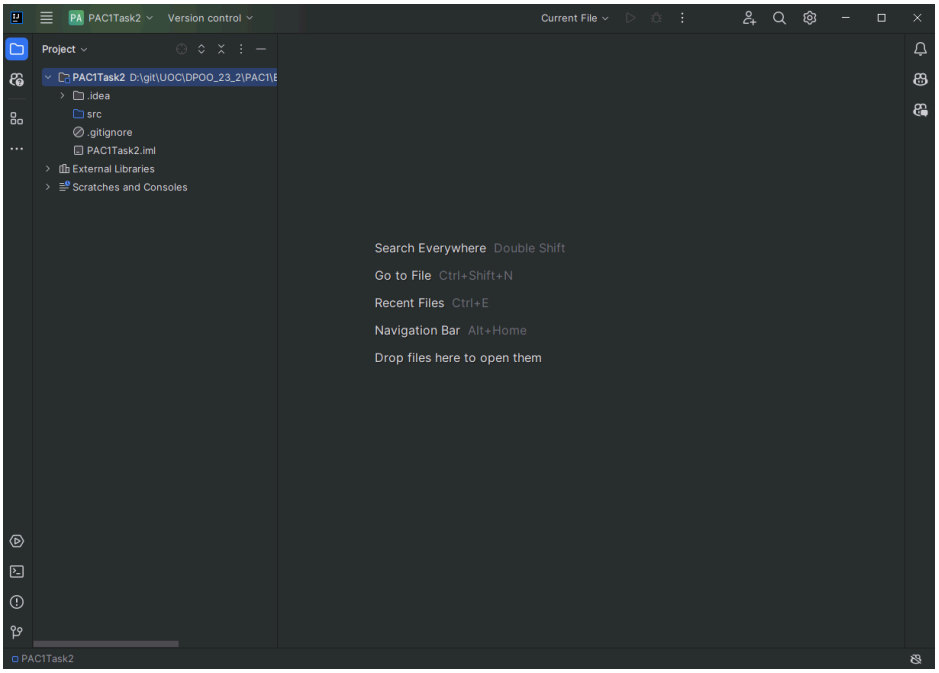
Una vez realizado el paso anterior, **ejecuta IntelliJ y crea un nuevo proyecto** con el botón `New Project`**:**



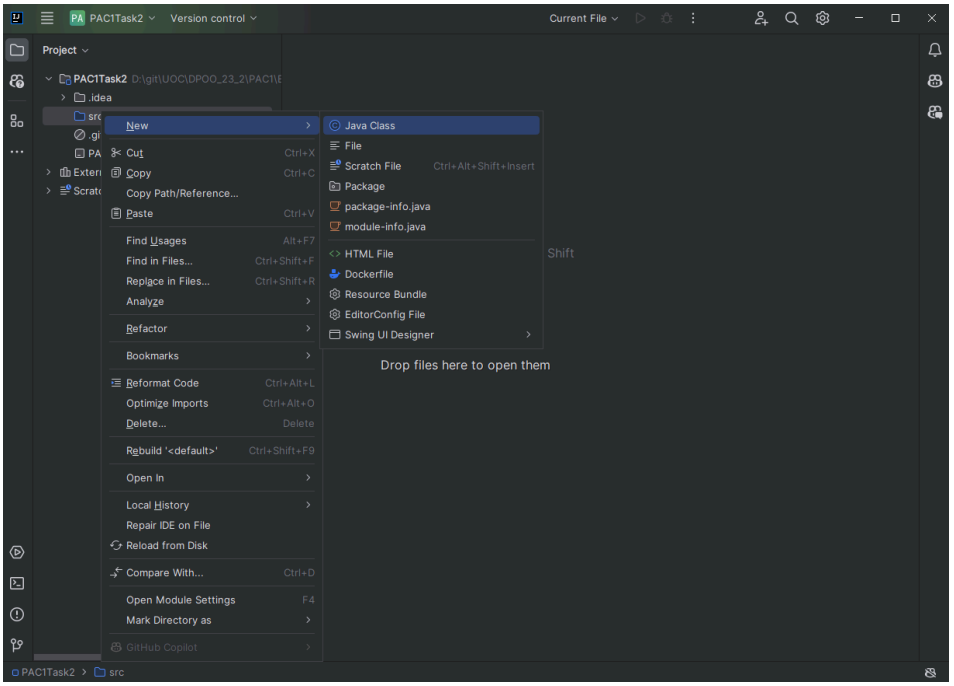
Una vez clicado el botón para crear un nuevo proyecto, IntelliJ te mostrará una ventana de configuración del proyecto. En primer lugar, debemos introducir el nombre del proyecto, en este caso, **`PAC1Task2`**, y seleccionar la carpeta que hará de *workspace* (i.e. espacio de trabajo) de tus proyectos. IntelliJ ya te aconseja un directorio y un nombre para tu espacio de trabajo (i.e. `IdeaProjects`). Además, IntelliJ selecciona por defecto como tipo de proyecto `Java` (lenguaje que usaremos durante toda la asignatura) y como SDK (*Software Development Kit*) el JDK que tienes instalado en tu máquina (JDK 21). Clicamos el botón `Create`.



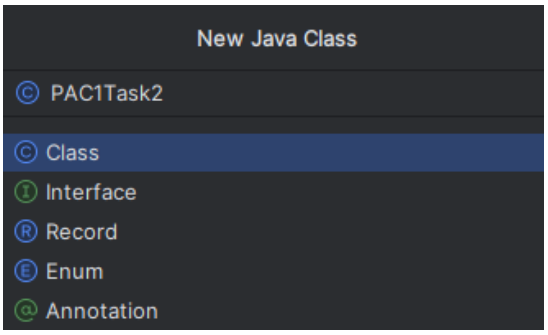
Tras crear el proyecto **`PAC1Task2`**, se abrirá el proyecto mostrando el *project tool window* en el lateral izquierdo. El *project tool window* muestra un árbol de directorios del proyecto. En él verás la carpeta `src` vacía. Esta carpeta/directorio será donde pondrás el código fuente de tu programa.



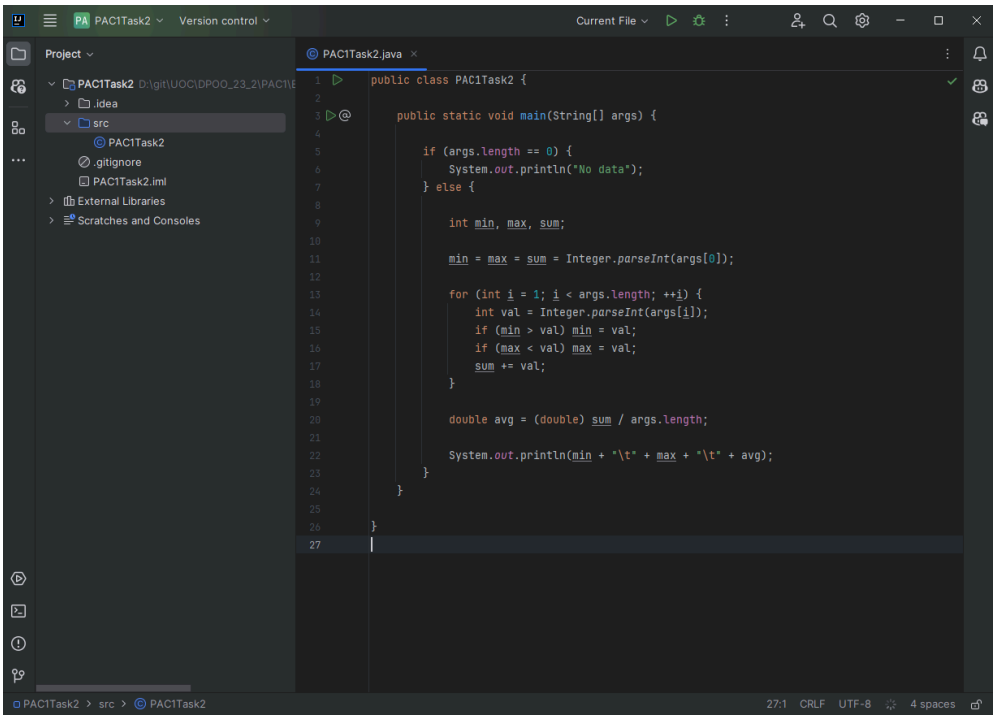
Haz clic con el botón derecho encima de la carpeta `src` y escoge la opción: `src → New → Java Class`.



En la ventana que aparecerá, introduce el nombre de la clase, en este caso, **`PAC1Task2`** (no hace falta ponerle la extensión `.java`, IntelliJ la pondrá por ti).



Una vez creada la clase, copia en el fichero `PAC1Task2.java` todo el código del método `main` (cabecera incluida) del fichero llamado `PAC1Task1.java` (puedes abrirlo con Notepad++) que encontrarás en la carpeta `PAC1Task1` que te hemos proporcionado con el enunciado y que has utilizado en la tarea anterior. Debería quedarte algo parecido a esto:



Truco: Si el código te aparece mal indentado (es decir, mal formateado), puedes hacer Ctrl+Alt+L (Windows) o Cmd+Alt+L (MacOS) para que IntelliJ indente el código correctamente.

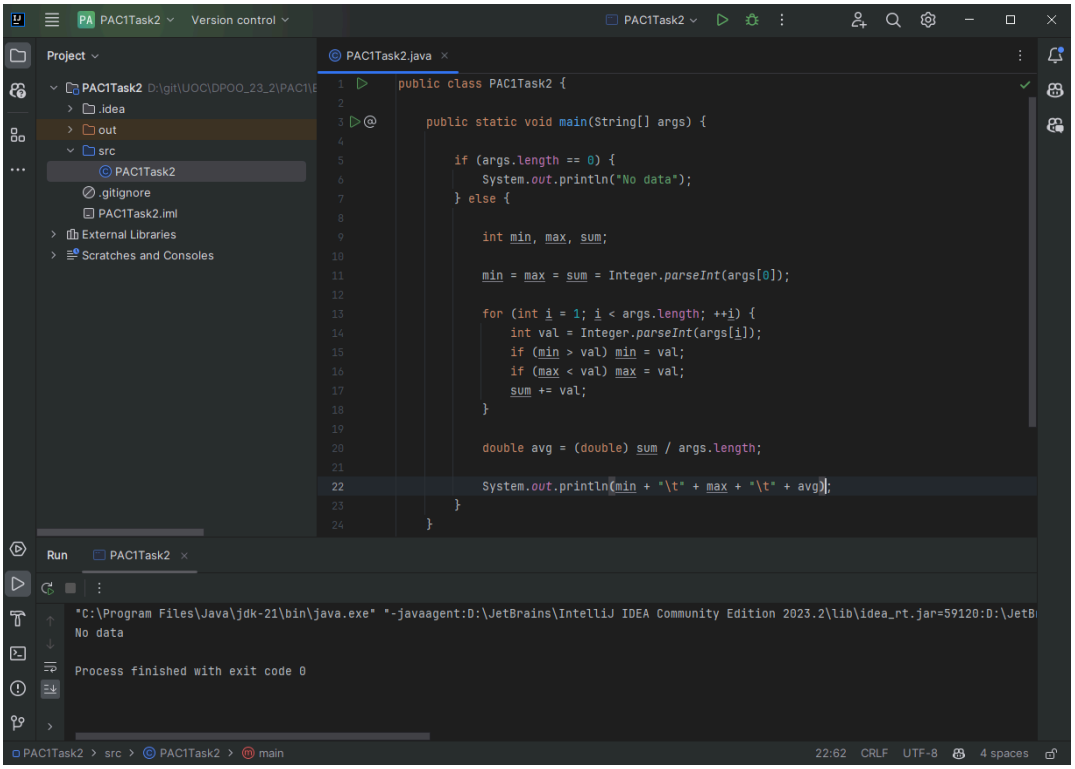
Si quieres saber más atajos (*shortcuts*) de IntelliJ, mira: https://www.jrebel.com/system/files/cheat-sheet-rebel-intellij-idea-web.pdf.

Ahora, en el *project tool window*, selecciona el fichero que acabas de crear `PAC1Task2.java` (la extensión `.java` no aparece visible en el *project tool window*), y haz clic con el botón derecho del mouse. En el menú contextual que aparecerá escoge la opción: `Run 'PAC1Task2.main()'`.

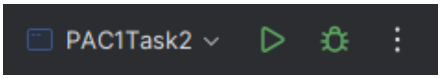
El programa será compilado (la primera vez puede tardar un poco) y, si no hay errores, se ejecutará. Fíjate en la parte inferior derecha de IntelliJ para ver qué está haciendo el IDE con tu proyecto. Indicará el estado en el que está, p.ej. *"Build"*, *"Parsing"*, *"Build completed"*, etc.



Una vez ejecutado, debe aparecerte la consola con el resultado `"No data"`, puesto que el programa no recibe ningún listado de números. Ahora el código de la Tarea 1 lo hemos transformado en un proyecto IntelliJ.

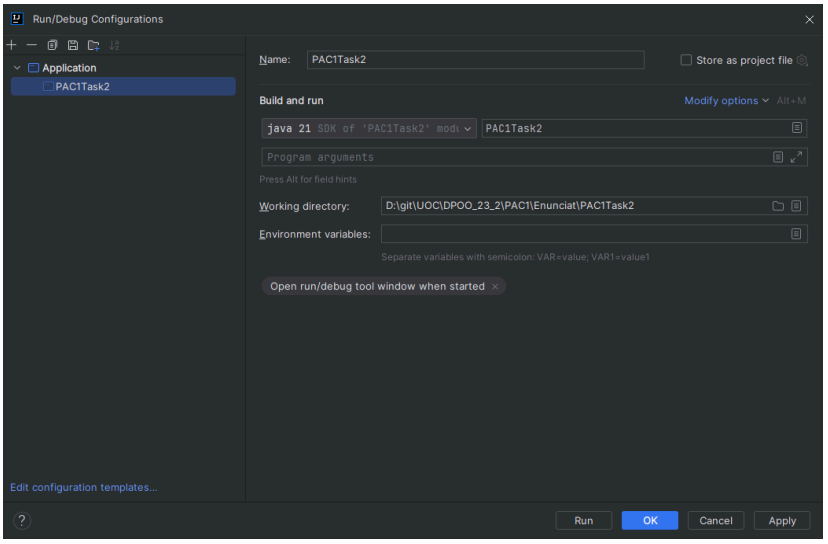


A continuación, fíjate que en la parte superior derecha aparece la información de la última ejecución que hemos lanzado. Ahora puedes lanzar la ejecución directamente desde el botón `Run`  .

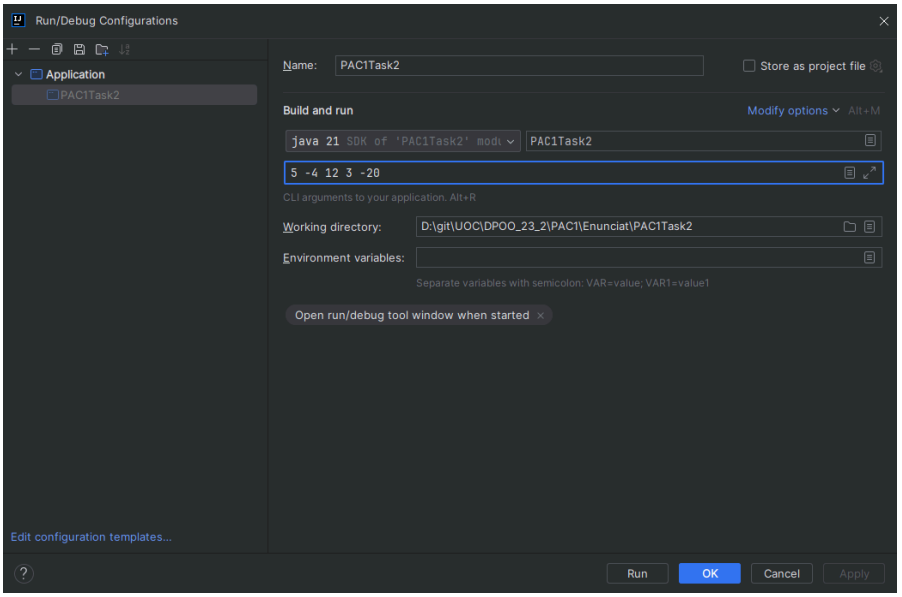


Si siempre das a `Run`, siempre te aparecerá `"No data"`. Es normal, puesto que no le estás pasando argumentos. ¿Cómo pasar argumentos a la ejecución?

1. Pulsa en el desplegable que hay junto al botón `Run` en el que pone `PAC1Task2` y verás que despliega un menú.
2. Haga clic en la opción `Edit...`
3. Se abrirá una ventana como la que se muestra a continuación.



4. En la caja de texto que pone `Program arguments`, escribe unos números separados por espacios.



5. Si ahora haces clic en el botón `OK` de la ventana y, posteriormente, en el botón `Run`  de la barra de ejecución, se ejecutará con los valores que has escrito como argumentos en `Program arguments`, siendo el resultado:

   `-20  12  -0.8`

   (si has puesto los números del ejemplo). Si quieres modificarlos, debes ir a `Edit...` y repetir el proceso.

Ejercicio 1 - Debugging (2 puntos)

Para realizar este ejercicio, copia la carpeta **`PAC1Ex1`** que te hemos proporcionado en tu carpeta *workspace* (normalmente llamada `IdeaProjects`). Si abres la carpeta/proyecto `PAC1Ex1` directamente con IntelliJ sin copiarla al *workspace*, entonces los cambios se verán reflejados en el proyecto que te hemos dado (puedes hacerlo, pero está bien tener una copia de seguridad y, al mismo tiempo, saber que todas tus entregas están dentro del *espacio de trabajo*).

Una vez abierto, verás que aparecen errores. Los errores que IntelliJ muestra, o bien son sintácticos, o bien hacen referencia a elementos que el IDE no conoce (estos errores también entran dentro del apartado sintáctico porque para IntelliJ son como si estuvieran mal escritos). Un error sintáctico también puede ser la ausencia o repetición de una llave `{}`, o que el orden de las palabras clave e identificadores no está bien (p. ej. `myName int;`), etc.

El programa proporcionado genera la secuencia correspondiente a la **conjetura de Collatz**: dado un número entero positivo `n`, el siguiente valor de la secuencia se calcula aplicando las reglas siguientes:

$$C(n) = n/2\,,\ si\ n\ es\ par$$

$$C(n) = 3n + 1\,,\ si\ n\ es\ impar$$

Este proceso se repite hasta que la secuencia alcanza el valor `1`. Por ejemplo, si el valor inicial es `6`, la secuencia obtenida debe ser:

```
6 → 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1
```

En este ejercicio, te pedimos:

a) Corrige, con la ayuda de IntelliJ, los errores sintácticos del programa de forma que no contenga ninguno.

(1 punto)

Ahora, si ejecutas el programa, verás que el resultado no es el esperado. ¿Por qué? Porque contiene errores de lógica. Es decir, la funcionalidad no está bien programada. Evidentemente, IntelliJ no es mágico y no sabe lo que pretendes programar. Así pues, debes comprobar mediante "trazas" (i.e. ejecutar el programa paso a paso) qué está haciendo el programa y detectar dónde están los problemas. Por eso puedes, o bien poner en ciertos puntos del programa instrucciones que impriman textos por pantalla (p. ej. usar `println`), o bien utilizar la funcionalidad de debugging (en español, depuración) que ofrece IntelliJ (ver el videotutorial que encontrarás en el bloque **UOCoders** del apartado Contenidos del aula, o el vídeo

https://www.youtube.com/watch?v=bb7Ny3M6cpY o el tutorial: https://www.jetbrains.com/help/idea/debugging-your-first-java-application.html).

Te recomendamos que le dediques algo de tiempo a aprender a debugar mínimamente con IntelliJ, ya que te será muy útil en futuras actividades (y en el mundo profesional).

b) Corrige los errores lógicos (también llamados errores semánticos) del programa de forma que se ejecute como se espera.

(1 punto)

Una vez corregidos todos los errores sintácticos y lógicos, la salida de vuestro programa debe tener el siguiente formato para estos dos ejemplos:

```
Enter a positive integer: 6
Collatz sequence: 6 -> 3 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
```

```
Enter a positive integer: 18
Collatz sequence: 18 -> 9 -> 28 -> 14 -> 7 -> 22 -> 11 -> 34 -> 17 -> 52 -> 26 -> 13 -> 40 -> 20 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
```

Requisito mínimo para evaluar este ejercicio: el programa no debe contener errores sintácticos (apartado A) ni semánticos (apartado B). Podéis validar los resultados obtenidos con los del pantallazo del ejemplo anterior.

Tarea 3 - Testing con JUnit

En el Ejercicio 1, la única manera que has tenido para saber si el programa se comportaba como era esperado ha sido ejecutándolo varias veces y viendo la salida por pantalla (o por el debugger). Realizar test y detectar errores utilizando el método del propio lenguaje que escribe por pantalla (p. ej., `printf`, `print`, `println` o el equivalente del lenguaje que utilices habitualmente) es una manera muy básica —muy de principiante—, aunque todo el mundo la suele utilizar en algún momento por su comodidad. Por su parte, la debugación (o depuración) se utiliza para detectar errores locales dentro del código.

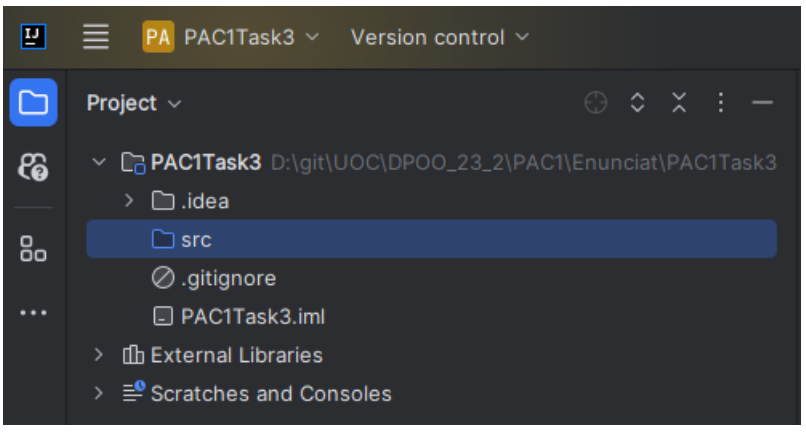
Dado que las pruebas son muy importantes para garantizar que un programa es correcto y se comporta como se espera, con el tiempo ha surgido una forma de desarrollar software basada justamente en el testeo. Es lo que se conoce como TDD o *Test Driven Development* (i.e. desarrollo conducido/dirigido por test). Esta forma de desarrollar tiene como lema: *"un programador debe escribir los casos de test antes que el código que estos tests comprobarán"*.

A partir de ahora, en esta asignatura escribirás o te proporcionaremos test para todos los ejercicios que requieren codificación. Para ello usaremos la librería JUnit. Es por este motivo que antes de continuar debes:

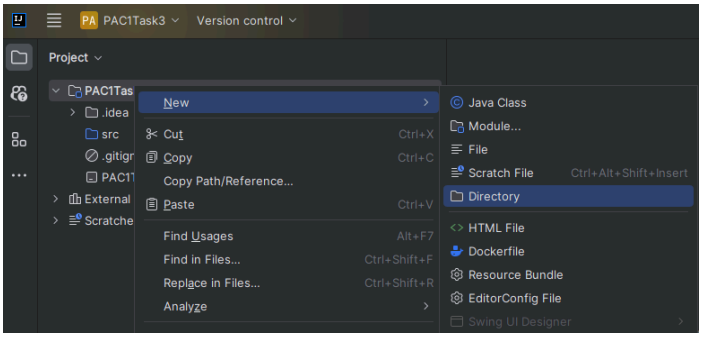
> Leer el apartado "**5.3.1. JUnit**" de la Guía de Java, que da una pequeña introducción sobre qué son los test unitarios.

Y, por otro lado, seguir los siguientes pasos para añadir, configurar y utilizar JUnit en un proyecto Java en el IDE IntelliJ (sin apoyo alguno de un gestor de dependencias como Maven o Gradle… esto ya lo trabajaremos en la PEC2).

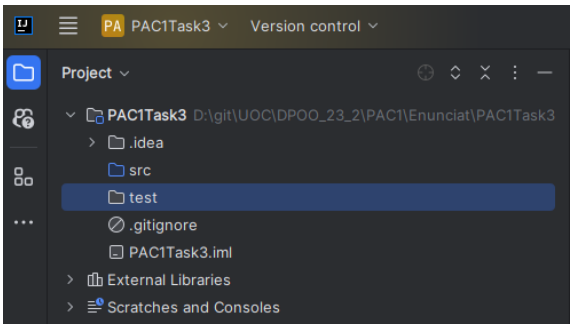
1. Crea un nuevo proyecto Java en IntelliJ desde `File → New → Project` llamado **`PAC1Task3`**.



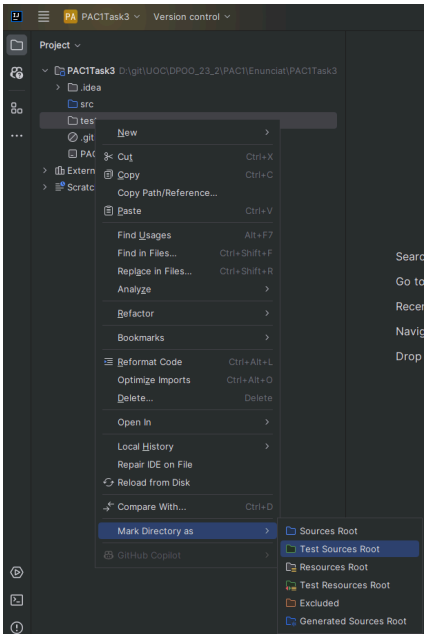
2. Desde la raíz del proyecto, crea, con el menú contextual que aparece al hacer clic en el botón derecho del mouse, un directorio (`Directory`) llamado `test` para almacenar los test unitarios JUnit.



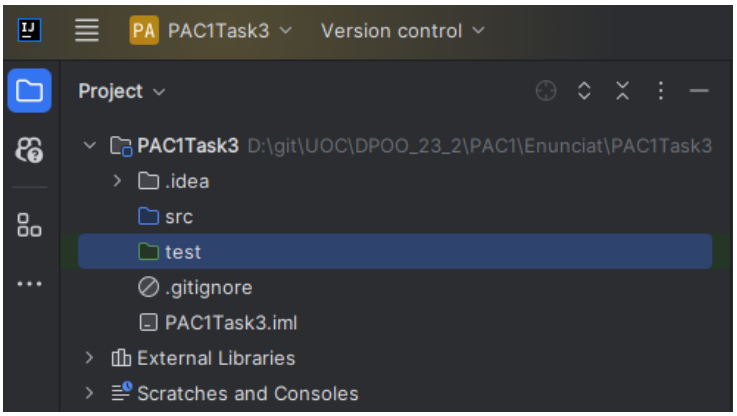
3. Una vez creado, verás en el *project tool window* una nueva carpeta `test`, al mismo nivel que la carpeta de código fuente `src`.



4. Selecciona la carpeta `test` y márcala como un directorio que será la raíz de las fuentes de los test unitarios (i.e. *Test Sources Root*). Para ello, haz clic con el botón derecho sobre el directorio `test` y escoge la opción `Mark Directory as → Test Sources Root`.



Verás que ha cambiado el color de la carpeta `test` a verde.



5. Una vez aquí, en el directorio `test`, crea la clase **`PAC1Task3Test`** donde implementarás los test unitarios. Ya has visto anteriormente cómo crear clases en los proyectos.

6. Crea la siguiente función/método dentro de esta clase:

```
public void testProduct() {
    assertEquals(42, PAC1Task3.product(6, 7));
}
```

7. Añade la anotación `@Test` encima de la firma del método. Con esto estamos especificando que se trata de un método de test JUnit. Además, esto nos permite importar desde IntelliJ la librería JUnit.

   Como verás, si pones el cursor encima de la línea `@Test`, te marca un error de IntelliJ.

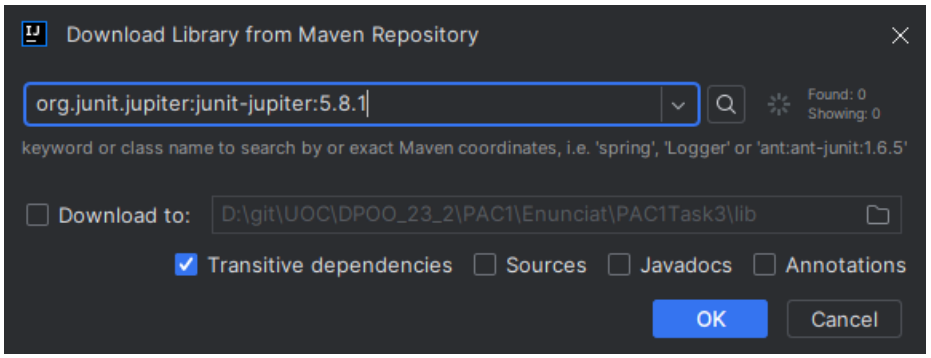


8. Clica encima de este globo rojo  e IntelliJ te proporcionará una manera fácil de añadir la librería JUnit 5 a tu proyecto.



9. En la siguiente pantalla, IntelliJ te permite descargar desde el repositorio Maven la librería JUnit 5.x (esto no significa que utilices Maven como gestor de dependencias, únicamente estás diciendo que descargue esta librería desde el repositorio oficial de Maven; podría ser desde otro repositorio o link).

   Puedes seleccionar el *check* `Download to` si quieres descargar los ficheros de la librería en una carpeta (`lib`) de tu proyecto, o no marcarlo si quieres añadir la librería como `External libraries`. En este caso utilizaremos la segunda opción.



10. Una vez añadida la librería JUnit, verás que ya no tienes el mismo error en la anotación `@Test`. Ahora, ya solo te queda resolver un último error, puesto que falta un `import` para poder utilizar las *assertions* (i.e. métodos/funciones) de JUnit. Por lo tanto, añade este `import` encima de la definición de la clase (si no lo ha hecho IntelliJ ya por ti):

```
import static org.junit.jupiter.api.Assertions.*;
```

11. Ahora, en el directorio `src`, crea la clase **`PAC1Task3`** donde tendrás la implementación del método `product` que has codificado en la clase de test **`PAC1Task3Test`**.

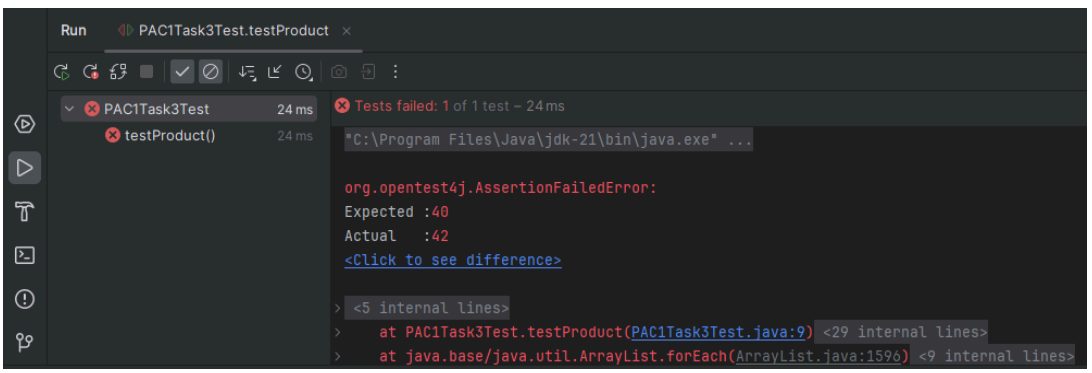
12. Dentro del fichero `PAC1Task3.java` escribe el siguiente código:

```
public class PAC1Task3 {
    public static int product(int a, int b){
        return a * b;
    }
}
```

13. Ahora ya puedes ejecutar el test implementado en la clase **`PAC1Task3Test`**. Puedes ejecutarlo como hemos explicado anteriormente, clicando en el botón derecho sobre la clase y accediendo a `Run`, o con los botones  o  que verás delante de la clase y los métodos que implementan los tests unitarios.



14. Modifica el valor de `expected` (`42`) para poner cualquier otro valor que sea incorrecto, vuelve a ejecutar y verás que el IDE te muestra los errores del JUnit de la forma: valor esperado y el actual, así como la traza de error.



Ejercicio 2 (3 puntos)

Una empresa de paquetería utiliza seis estaciones de taquillas (*lockers*) inteligentes para depositar los paquetes que esperan ser recogidos. Cada estación tiene una capacidad distinta. El programa que se te pide que realices debe analizar el grado de ocupación, detectar cuándo conviene retirar paquetes y generar un informe global.

Abre el proyecto **`PAC1Ex2`** e implementa los seis métodos marcados con `TODO` en `src/PAC1Ex2.java`.



Consejo: Ten en cuenta que el código base que te proporcionamos incluye una serie de variables (constantes) que debes utilizar:

- `LOCKER_CAPACITIES` almacena el número de paquetes que admite cada estación.
- `STATION_NAMES` almacena los nombres de las estaciones.
- `COLLECTION_THRESHOLD` indica la tasa de ocupación a partir de la cual es necesario recoger paquetes.

Debes codificar las siguientes funciones:

- **`getLockerCapacity`**: Devuelve la capacidad almacenada en `LOCKER_CAPACITIES` para el índice indicado. Se asume que el índice es válido, por lo que no debes añadir validaciones ni valores especiales de error.

  (0.20 puntos test proporcionado)

- **`calculateOccupancyRate`**: Devuelve la tasa de ocupación como el cociente real entre `packages` y la capacidad de la estación (`LOCKER_CAPACITIES`), sin convertirla en porcentaje ni redondearla.

  (0.20 puntos test proporcionado)

- **`needsPackageCollection`**: Devuelve `true` si la tasa calculada es mayor o igual que la tasa de ocupación (`COLLECTION_THRESHOLD`) y `false` si es inferior. El número de paquetes puede ser cero o superar la capacidad.

  (0.20 puntos test proporcionado)

  En la siguiente tabla te proporcionamos varios ejemplos de qué debe devolver cada una de las tres funciones anteriores en función de los valores de entrada, suponiendo un `COLLECTION_THRESHOLD = 0.80`.

Packages	*Station*	*getLockerCapacity*	*calculateOccupancyRate*	*needsPackageCollection*
0	0	40	0.0	false
31	0	40	0.775	false
32	0	40	0.8	true
48	2	60	0.8	true
28	3	32	0.875	true
60	4	48	1.25	true

- **`calculateTotalPackages`**: Devuelve el número total de paquetes almacenados en todas las estaciones.

(0.40 punto test proporcionado)

- **`getBusiestStation`**: Devuelve el índice de la estación con la tasa de ocupación más alta. Debes comparar tasas calculadas con la capacidad correspondiente, no cantidades absolutas de paquetes. Si varias estaciones tienen la misma tasa máxima, devuelve el índice menor. Se asume que el *array* contiene las seis estaciones. Así pues, si todas las estaciones están al 50%, el resultado debe ser 0.



Consejo: Para resolver los empates, inicializa el resultado con la primera estación y sustitúyelo solo cuando encuentres una tasa estrictamente mayor.

(1.50 puntos test proporcionado)

- **`printLockerReport:`** Imprime el número total de paquetes (`"Total packages:"`) y el nombre de la estación con mayor ocupación (`"Busiest station:"`). Ambas líneas deben terminar con un salto de línea. No añadas líneas vacías ni mensajes adicionales. El método no debe modificar el *array* recibido.

(0.50 puntos test proporcionado)



Requisito mínimo para evaluar este ejercicio: el programa debe pasar todos los test proporcionados para **`calculateOccupancyRate`**, **`needsPackageCollection`**, **`calculateTotalPackages`** y **`getBusiestStation`**.



En la corrección se valorará también la calidad del código y se podrá descontar hasta 1 punto del total. Se tendrá en cuenta, entre otros aspectos, la claridad del código, los nombres de las variables, el uso de constantes y la reutilización de métodos.

Ejercicio 3 (5 puntos)

Copia en tu workspace el proyecto **`PAC1Ex3`** facilitado con el enunciado y ábrelo con IntelliJ. En este ejercicio programarás varios métodos que ayudan a una asociación vecinal a gestionar las solicitudes de parcelas de su huerto comunitario.

Cada grupo solicita un número de parcelas para cultivarlas durante una temporada. Para facilitar el trabajo conjunto, todas las parcelas del grupo deben estar disponibles y ser consecutivas en una misma fila. El programa debe localizar el primer bloque que cumpla estas condiciones y calcular su cuota de temporada.

Algunas parcelas ya están asignadas y otras se mantienen en descanso para recuperar el suelo (barbecho). Ninguna de estas parcelas puede ser parte de una nueva solicitud.

En la clase **`PAC1Ex3.java`**, el parámetro **`garden`** es una matriz de enteros donde cada celda representa una parcela del huerto y contiene su estado (`AVAILABLE`, `ASSIGNED` o `RESTING`). Las filas y las columnas se numeran desde cero: `garden[row][column]` contiene el estado de la parcela situada en la fila `row` y la columna `column`. Las dimensiones se obtienen de la propia matriz:

- `garden.length`: número de filas del huerto.
- `garden[0].length`: número de columnas del huerto.

Por tanto, el número total de parcelas es `garden.length * garden[0].length`.

En el ejemplo de este enunciado, el huerto tiene 4 filas y 8 columnas: 32 parcelas.

En la clase **`PAC1Ex3.java`** encontrarás las siguientes constantes que debes usar:

- `AVAILABLE`: parcela disponible.
- `ASSIGNED`: parcela ya asignada.
- `RESTING`: parcela en descanso.
- `BASE_PRICE`: cuota base, en euros, de una parcela.
- `IRRIGATED_ROWS`: número de filas iniciales que tienen riego instalado. Con el valor actual que hay en el código, las filas `0` y `1` serían las que tienen riego instalado, ya que el valor de `IRRIGATED_ROWS` es `2`.
- `IRRIGATION_SURCHARGE`: suplemento, en euros, por parcela y temporada que se añade a la cuota base en las filas con riego.
- `DISCOUNT_MIN_PLOTS`: número mínimo de parcelas de una solicitud para obtener el descuento de grupo.
- `GROUP_DISCOUNT`: proporción de descuento que se aplica al importe total cuando la solicitud alcanza ese mínimo. El valor `0.15` equivale al `15%` e incluye el descuento sobre el suplemento de riego, si corresponde.

Puedes asumir que la matriz:

- No es nula.
- Es rectangular.
- Tiene al menos una fila y una columna.
- No contiene filas nulas y el valor de las casillas (parcelas) solo puede ser uno de los siguientes tres estados: `AVAILABLE` (valor 0), `ASSIGNED` (valor 1) o `RESTING` (valor 2).

No debes validar las condiciones anteriores. Las dimensiones de la matriz pueden cambiar entre llamadas: debes obtenerlas del propio array. En cambio, los índices y la cantidad de parcelas solicitadas sí pueden ser inválidos, y debes tratarlos según los contratos de los métodos.

Ejemplo (utilizado en los tests):

1	0	0	2	0	0	0	1
0	0	1	0	0	0	0	2
0	0	0	0	0	1	0	0
2	0	0	0	0	0	0	0

La fila `0` contiene cinco parcelas disponibles, pero no permite ubicar cuatro parcelas consecutivas. Sí permite ubicar tres, empezando por la columna `4`. La parcela en descanso de la columna `3` interrumpe la continuidad del bloque.

A continuación, debes codificar los `TODO` (del inglés, *"to do"*, en español, "por hacer"). Este ejercicio consiste básicamente en implementar los 5 métodos/funciones siguientes:

- **`isValidPlot(int[][] garden, int row, int column)`**: devuelve `true` si `row` y `column` identifican una posición existente de la matriz, y `false` en caso contrario. Es decir, devuelve `true` si existe la casilla. Esto es independiente del estado de la parcela. Este método comprueba exclusivamente los índices.

  (0.5 puntos: 0.5 puntos tests proporcionados)

- **`countAvailablePlots(int[][] garden, int row)`**: devuelve el número de parcelas con estado `AVAILABLE` en la fila indicada, aunque estén separadas entre sí. Si la fila es inválida, devuelve `-1`. Si la fila existe, pero no tiene parcelas disponibles, devuelve `0`.

  (0.5 puntos: 0.5 puntos tests proporcionados)

- **`canAssignPlots(int[][] garden, int row, int startColumn, int plotCount)`**: devuelve `true` si desde la columna `startColumn` (incluida) de la fila `row` de `garden` hay un total de `plotCount` parcelas consecutivas disponibles. Devuelve `false` si la posición inicial es inválida, `plotCount` es cero o negativo, el número de parcelas consecutivas que se solicita rebasa la longitud de la fila o si no hay `plotCount` parcelas consecutivas disponibles.



Consejo: Comprueba los límites antes de acceder al array. Reutiliza `isValidPlot`.

(1.0 puntos: 1.0 puntos tests proporcionados)

- **`findPlotBlock(int[][] garden, int plotCount)`**: devuelve un *array* con 2 valores enteros: `{fila, columnaInicial}` correspondiente a la posición del primer bloque con un número `plotCount` de parcelas consecutivas disponibles. Si `plotCount` no es positivo, supera el número de columnas o no existe un bloque adecuado, devuelve un array con los valores `{-1, -1}`.



Consejo: La búsqueda se realiza recorriendo las filas desde el índice `0` hasta la última y, dentro de cada fila, las columnas de izquierda a derecha desde el índice `0`. Reutiliza `isValidPlot` y `canAssignPlots`.

(1.5 puntos: 1.5 puntos tests proporcionados)

- **`calculateLeasePrice(int[][] garden, int plotCount)`**: obtiene la ubicación del primer bloque de parcelas que contiene `plotCount` parcelas mediante `findPlotBlock` y devuelve la cuota total que supone alquilar el bloque de parcelas solicitada. Si no existe una ubicación válida, devuelve `-1.0`. Las tarifas ficticias que se aplican en el programa de este ejercicio son:
  - La cuota base es de `25,00 EUR` por parcela (`BASE_PRICE = 25.00`).
  - Si las parcelas del bloque encontrado están ubicadas en las filas con índice menor que `IRRIGATED_ROWS = 2`, esto significa que dichas parcelas tienen riego instalado. Cada parcela de estas filas tiene un suplemento de `7,50 EUR` (`IRRIGATION_SURCHARGE = 7.50`).
  - Si el número de parcelas (`plotCount`) que se quiere alquilar en un bloque es de al menos `DISCOUNT_MIN_PLOTS = 4` parcelas, entonces el precio total del alquiler recibe un descuento del `15%` (`GROUP_DISCOUNT = 0.15`).

  Primero calcula la tarifa de una parcela según la fila elegida, después multiplícala por `plotCount` y, si corresponde, aplica el descuento sobre todo el importe, incluido el suplemento.



Consejo: Redondea únicamente el resultado final a dos decimales mediante `Math.round(value * 100.0) / 100.0`. No redondees los cálculos intermedios. El resultado es un número: no es necesario formatearlo como texto.

Igualmente, reutiliza funciones codificadas previamente.

Siguiendo el ejemplo del huerto anterior:

plotCount	findPlotBlock	calculateLeasePrice (EUR)
1	{0, 1}	32.50
2	{0, 1}	65.00
3	{0, 4}	97.50
4	{1, 3}	110.50
5	{2, 0}	106.25
6	{3, 1}	127.50
7	{3, 1}	148.75
8	{-1, -1}	-1.0
0	{-1, -1}	-1.0

(1.5 puntos: 1.5 puntos tests proporcionados)



Importante: Ninguno de los métodos de este ejercicio debe modificar el contenido de la matriz `garden`. Los métodos únicamente consultan su estado y devuelven el resultado correspondiente.



Requisito mínimo para evaluar este ejercicio: el programa debe compilar y superar todos los tests proporcionados para las funciones: **`isValidPlot`**, **`countAvailablePlots`**, **`canAssignPlots`** y **`findPlotBlock`**



En la corrección se valorará también la calidad del código y se podrá descontar hasta 1 punto del total. Se tendrá en cuenta, entre otros aspectos, la claridad del código, los nombres de las variables, el uso de constantes y la reutilización de métodos.

Formato y fecha de entrega

Tienes que entregar un archivo *.zip, cuyo nombre debe seguir este patrón: `loginUOC_PEC1.zip`. Por ejemplo: `dgarciaso_PEC1.zip`. Este archivo comprimido debe incluir los siguientes elementos:

- El proyecto IntelliJ `PAC1Ex1` completado siguiendo las peticiones y especificaciones del Ejercicio 1.
- El proyecto completo IntelliJ `PAC1Ex2` completado siguiendo las peticiones y especificaciones del Ejercicio 2.
- El proyecto completo IntelliJ `PAC1Ex3` completado siguiendo las peticiones y especificaciones del Ejercicio 3.

El último día para entregar esta PEC es el **06/10/2026** antes de las 23:59. Cualquier PEC entregada más tarde será considerada como no presentada.



No respetar el formato de entrega de un ejercicio supondrá la no corrección del mismo y, por tanto, una calificación de cero puntos en ese ejercicio.



El Equipo Docente podrá solicitar a cualquier estudiante que explique, justifique o modifique el código entregado, con el fin de verificar su autoría y garantizar que comprende adecuadamente la solución presentada. La imposibilidad de justificar de forma satisfactoria la solución podrá dar lugar a las actuaciones previstas en la normativa de integridad académica de la Universidad.

El medio utilizado para dicha justificación podrá ser diverso, en función de las circunstancias: entrevista oral síncrona mediante videoconferencia, comunicación por correo electrónico, prueba escrita síncrona o asíncrona, o cualquier otro procedimiento que permita verificar de manera adecuada la autoría y comprensión de la entrega.

Anexo: Problemas frecuentes

En este capítulo, se presentan algunos de los problemas más frecuentes observados durante la creación y ejecución de proyectos en IntelliJ relacionados con la PEC1. Si os encontráis con problemas no resueltos en este anexo, podéis utilizar el foro de la asignatura para resolverlos.

Tened en cuenta que los problemas que incluye este anexo pueden persistir en las futuras actividades de la asignatura. Por lo tanto, es altamente recomendable que apliquéis las soluciones propuestas a lo largo de la asignatura y no solo en esta actividad.

Uso de servicios de almacenamiento en la nube

En algunas acciones, como la limpieza de los archivos generados durante la compilación de un proyecto, los servicios de almacenamiento en la nube como Google Drive, OneDrive o Dropbox pueden provocar problemas, especialmente si se tiene activada la opción de sincronización automática. Concretamente, es posible que os aparecen errores en los que se indique que no se pueden eliminar los archivos temporales como el siguiente:

```
unable to delete directory....\PAC1Ex3\build\test-results\test\binary
```

Si queréis utilizar estos servicios, lo recomendable es desactivar la sincronización automática o excluir la carpeta del proyecto mientras trabajáis, para volver a añadirla después.

En cualquier caso, podéis utilizar herramientas como Git como alternativa para almacenar el código fuente de vuestro proyecto en un repositorio. Con esta herramienta, no os debería dar ningún problema. Además, Git es muy utilizado en entornos profesionales, por lo que podéis aprovechar para empezar a familiarizaros con él. No obstante, debéis tener en cuenta que en la asignatura no se ofrecerá soporte técnico con el uso de estas herramientas, ya que quedan fuera del alcance de la asignatura.